



LOOP

(Live Object-Oriented Playground)

Personal Project 03

Jun Xiang

4th October 2026

Table of Contents

1. Overview	3
1.1 Problem	3
1.2 Solution	3
1.3 Components	Error! Bookmark not defined.
1.4 Features.....	4
2. Scope.....	4
2.1 In Scope	4
2.2 Out of Scope.....	5
3. Setup & Design.....	Error! Bookmark not defined.
4. Methodology	7
5. Limitations & Risks	10
6. Testings and Findings.....	11
7. Challenges	19
8. Improvements.....	19

1. Overview

LOOP (Live Object-Oriented Playground) is an interactive Python OOP visualizer designed to make object-oriented programming concepts easier to understand through visual execution.

Currently, it uses a fictional cybersecurity scenario where users can interact with attackers, network devices, and defenders. As Python methods execute, LOOP visually shows method calls, object interactions, state changes, inheritance, and the corresponding source code in real time.

1.1 Problem

Object-oriented programming concepts such as classes, objects, inheritance, encapsulation, and object interaction can be difficult to understand from source code alone.

Beginners can see what the code says, but may not clearly see which object is running a method, which object is affected, or how its state changes during execution.

1.2 Solution

LOOP (Live Object-Oriented Playground) is an interactive Python OOP visualizer.

It uses scenario-based simulations to show how Python objects interact during execution. The current scenario is a fictional cybersecurity environment containing an attacker, network devices, connections, and a defender.

Users can click methods or use Guided Mode to watch:

- real Python methods execute through FastAPI
- source code highlight during execution
- object instances become active
- interactions animate between objects
- object state change
- inherited methods execute through `super()`
- execution events appear in the Terminal

The cybersecurity behaviour is fictional and does not interact with real systems or networks.

1.3 Goals

LOOP aims to:

- Make Python OOP concepts easier to understand visually.
- Clearly distinguish classes from object instances.
- Show method calls and object interactions during execution.
- Demonstrate encapsulation, inheritance, composition, abstraction, and polymorphism.
- Keep Python as the source of truth for simulation state.
- Support different learning scenarios in the future.

1.4 Features

- Manual and Guided simulation modes.
- Real Python execution through FastAPI.
- Six fictional attack types.
- Defensive actions such as block, isolate, restore, and secure.
- Real Python source-code display.
- Source-line highlighting.
- Object-instance indicators.
- Animated interaction paths.
- Real before-and-after state changes.
- Terminal execution history with OOP explanations.
- Previous, Next, Play, Pause, Replay, and Reset controls.
- Visual inheritance and `super()` calls.

2. Scope

2.1 In Scope

Simulation

- One attacker, one defender, two network devices, and one connection.
- Manual and Guided modes.
- Fictional attack and defense actions.
- Repeatable simulations with no scores, timers, or win/loss conditions.

Object-Oriented Programming

- Classes and object instances.
- Encapsulation of object state.
- Composition between simulation objects.
- Abstract classes and `@abstractmethod`.
- Inheritance and method overriding.
- Use of `super()`.
- Polymorphism through a common `execute()` method.

Visualization

- Real Python source code display.
- Interactive method blocks.
- Source-line highlighting during execution.
- Object-instance indicators such as `DEV001`, `DEV002`, and `CON001`.
- Animated interaction paths between objects.

- Real before-and-after state changes.
- Visual inheritance and parent-method calls.
- Terminal history showing execution events and OOP explanations.

User Interface

- HTML, CSS, and JavaScript frontend.
- Attacker, Victim / Device, and Defender panels.
- Manual and Guided mode controls.
- Previous, Next, Play, Pause, Replay, and Reset controls.

2.2 Out of Scope

- Real cybersecurity attacks or network access.
- Real malware, ransomware, phishing, or DoS activity.
- Password cracking or credential collection.
- Vulnerability scanning or packet capture.
- Real firewall or antivirus integration.
- User accounts or authentication.
- Databases or cloud deployment.
- Multiplayer features.
- AI or machine learning.
- Full Python debugging or interpreter-level tracing.

3.1 Functional Requirements

No.	Non-Functional Requirement	Description
FR-01	Simulation Control	Users can play, pause, step through, replay, and reset a simulation.
FR-02	Manual Mode	Users can select attack and defense methods directly.
FR-03	Guided Mode	The system demonstrates all six attack types in a guided sequence.
FR-04	Python Execution	Selected actions execute the real Python simulation through FastAPI.
FR-05	Source Code Display	Users can view the real Python source code for each method.
FR-06	Execution Visualization	Executing methods, source lines, and interaction paths are visually highlighted.
FR-07	Object Instances	Actual objects such as DEV001, DEV002, and CON001 are shown separately from their classes.
FR-08	State Changes	Real before-and-after state values are displayed during execution.
FR-09	Inheritance Visualization	Inherited calls such as DDoSAttack → DoSAttack are shown visually.
FR-10	Execution Terminal	A scrolling Terminal shows execution history, descriptions, code explanations, and OOP concepts.

3.2 Non-Functional Requirements

No.	Non-Functional Requirement	Description
NFR-01	Safety	LOOP must not interact with real networks or systems.
NFR-02	Simplicity	The interface should use clear labels, limited controls, and short explanations to reduce unnecessary complexity.
NFR-03	Maintainability	Python logic, JavaScript behaviour, and CSS styling should remain clearly separated.
NFR-04	Single Source of Truth	Python objects contain the authoritative simulation state.
NFR-05	Readability	Displayed code and explanations should remain concise and understandable.
NFR-06	Extensibility	New attacks, devices, and scenarios should be addable without major redesign.
NFR-07	Local Execution	LOOP should require minimal setup to run locally.

4. Methodology

4.1 Technology Stack

- Python: OOP classes, simulation logic, object state, and execution tracing.
- HTML: Interface structure and source-code areas.
- CSS: Handles layout, highlighting, animations, and visual effects.
- JavaScript: Handles user interaction, playback, source highlighting, and visualization
- FastAPI: Lightweight connection between Python and the browser.

4.2 Running the Project

From the project folder: `python -m uvicorn api:app --reload`

Then open: `http://127.0.0.1:8000`

4.3 Application Structure

- LOOP separates the simulation from the visualization:
- Python Simulation → ExecutionTracer → FastAPI → JavaScript Playback → Browser Visualization
- Python contains the real object state and behaviour.
- JavaScript receives the execution events and uses them to control the visual interface.

4.4 Main OOP Design

Class	Purpose
CyberDefenseSimulation	Holds the simulation objects and coordinates actions
Attacker	Performs attacks using Attack objects
Defender	Performs defensive actions
Attack	Abstract base class defining execute()
NetworkDevice	Stores and manages device state
Connection	Represents a connection between devices
ExecutionTracer	Records important execution events
ExecutionEvent	Stores information about one execution step

Attack subclasses include:

- CredentialAttack
- MitMAttack
- DoSAttack
- DDoSAttack
- MalwareAttack
- RansomwareAttack

4.5 OOP Relationships

LOOP demonstrates several OOP concepts:

- Encapsulation: objects update their own internal state through methods.
- Composition: the simulation contains attackers, defenders, devices, and connections.
- Abstraction: `Attack` defines a common `execute()` interface.
- Polymorphism: different attack objects use the same `execute()` method.

- Inheritance: `DDoSAttack` extends `DoSAttack`, while `RansomwareAttack` extends `MalwareAttack`.
- Method overriding: child classes provide their own version of `execute()`.
- `super()`: child methods reuse behaviour from their parent class.

4.6 Execution Flow

When a method is selected:

1. JavaScript sends the request to FastAPI.
2. FastAPI executes the real Python method.
3. ExecutionTracer records the method calls and state changes.
4. The events are returned to the browser.
5. JavaScript plays the events one at a time.
6. Source code, object instances, interaction paths, and state changes are highlighted.
7. The Terminal records the execution history and explains the OOP concepts involved.

4.7 Manual and Guided Modes

- Manual Mode: Users can select methods directly and explore the simulation freely.
- Guided Mode: LOOP demonstrates the six attack types in sequence. Each scene focuses on specific OOP concepts and starts from a predictable simulation state.

4.8 Design Decisions

- Python is the source of truth. The browser does not decide the simulation state.
- Structured execution events. LOOP records meaningful events instead of tracing every Python operation.
- Classes and objects are shown separately. Object chips such as `DEV001` and `CON001` help distinguish instances from classes.
- Inheritance is shown visually. Calls through `super()` are highlighted so users can see when parent methods execute.
- Cybersecurity is only the scenario. The design can support other scenarios in the future.

5. Limitations & Risks

- The cybersecurity behaviour is simplified and does not represent real-world attacks.
- The project focuses on selected OOP concepts rather than covering all of Python OOP.
- Too much visual information at once may confuse users.
- Browser animation timing may not perfectly match Python execution timing.
- The frontend must remain synchronized with the real Python object state.
- Source-code explanations may need to be simplified for readability.
- The current version only includes one scenario: cybersecurity.
- The interface is designed mainly for desktop use and may require further adjustment for smaller screens.

6. Testings and Findings

6.1 Test Case 1: Guided Mode Interface

Test Input	Expected Result	Findings
Switch from Manual Mode to Guided Mode.	Guided Mode should become active and display the current scene information. "Play" button should be enabled to play different scenes.	- Guided Mode was activated successfully. - The current scene title, focus, and status were displayed. - Manual method execution was restricted while Guided Mode was active.

Test Evidence

LOOP

PLAYBACKPreviousNextPlayPauseReplayReset0 / 0

SCENARIO Cybersecurity

TERMINAL

MODEManualGuided

EVENT 0 / 0

SCENE 1 / 6 Credential Attack Focus: Object Interaction · Encapsulation

Press Play to begin.

Credential Attack · Ready

Watch CredentialAttack call a NetworkDevice method instead of changing the device's private state directly.

Press Play to run the real Python scene, then use Previous or Next to review its events.

ATTACKER01

Attacker

OBJECT [Attacker 1]

CLASS

> perform_attack()

CredentialAttack

OBJECT [Credential Attack]

CLASS

> execute()

MitMAttack

OBJECT [MITM Attack]

CLASS

> execute()

DoSAttack

OBJECT [DoS Attack]

CLASS

> execute()

↳ inherits DoSAttack

DDoSAttack

OBJECT [DDoS Attack]

CLASS

> execute()

MalwareAttack

OBJECT [Malware Attack]

CLASS

> execute()

↳ inherits MalwareAttack

RansomwareAttack

OBJECT [Ransomware Attack]

CLASS

> execute()

VICTIM / DEVICE02

NetworkDevice

OBJECT [DEV001] [DEV002]

CLASS

> increase_risk()

> overload()

> isolate()

> block()

> unblock()

> lock_files()

> restore()

Connection

OBJECT [CON001]

CLASS

> intercept()

> secure()

> block()

> unblock()

DEFENDER03

Defender

OBJECT [Defender 1]

CLASS

> inspect()

> block()

> unblock()

> isolate()

> restore()

> secure_connection()

GUIDED READY

6.2 Test Case 2: Manual Mode Interface

Test Input	Expected Result	Findings
Switch from Guided Mode to Manual Mode.	The normal interactive interface should return and methods should be selectable directly. All buttons should be disabled except “Reset” until one of the class method is pressed.	- Manual Mode was activated successfully. - Attack and defense methods were available for direct interaction. - Guided scene information was hidden.

Test Evidence

The screenshot displays the 'Manual' mode interface of a cybersecurity simulation. At the top, a 'PLAYBACK' bar includes buttons for 'Previous', 'Next', 'Play', 'Pause', 'Replay', and 'Reset', along with a progress indicator '0 / 0'. The 'SCENARIO' is set to 'Cybersecurity'. Below this, a 'TERMINAL' window shows the text: 'Waiting for execution...' and 'Select a method to begin. The execution history will appear here.' The main interface is divided into three columns: 'ATTACKER' (01), 'VICTIM / DEVICE' (02), and 'DEFENDER' (03). The 'ATTACKER' column lists classes: 'Attacker' (with 'perform_attack()' method), 'CredentialAttack' (with 'execute()' method), 'MitMAttack' (with 'execute()' method), 'DoSAttack' (with 'execute()' method), 'DDoSAttack' (inheriting from DoSAttack, with 'execute()' method), 'MalwareAttack' (with 'execute()' method), and 'RansomwareAttack' (inheriting from MalwareAttack, with 'execute()' method). The 'VICTIM / DEVICE' column lists classes: 'NetworkDevice' (with 'increase_risk()', 'overload()', 'isolate()', 'block()', 'unblock()', and 'lock_files()' methods), 'Connection' (with 'intercept()', 'secure()', 'block()', and 'unblock()' methods), and 'Defender' (with 'inspect()', 'block()', 'unblock()', 'isolate()', 'restore()', and 'secure_connection()' methods). The 'DEFENDER' column is currently empty. The bottom right corner indicates 'SYSTEM READY'.

6.3 Test Case 3: Credential Attack

Test Input	Expected Result	Findings
Click <code>execute()</code> under <code>CredentialAttack</code> .	Simulation should start running. <code>DEV001</code> should be targeted and its risk level should increase.	<code>DEV001</code> was highlighted as the affected <code>object.risk_level</code> changed from 0 → 1. - The interaction path and affected object were highlighted. - The relevant Python source lines and Terminal events were shown.

Test Evidence

The screenshot displays the Cybersecurity simulation interface. At the top, there's a 'LOOP' section with playback controls (Previous, Next, Play, Pause, Replay, Reset) and a 'SCENARIO Cybersecurity' label. Below this is a 'TERMINAL' window showing the following output:

```

03 STATE DEV001 · risk_level
0 → 1

04 DONE NetworkDevice.increase_risk()
DEV001

05 DONE CredentialAttack.execute()
Credential Attack → DEV001

```

The main area shows a class hierarchy with three columns: ATTACKER, VICTIM / DEVICE, and DEFENDER.

- ATTACKER (01):** Contains classes like `Attacker`, `CredentialAttack`, `MitMAttack`, `DoSAttack`, `DDoSAttack`, `MalwareAttack`, and `RansomwareAttack`. The `CredentialAttack` class is highlighted, showing its `execute()` method.
- VICTIM / DEVICE (02):** Contains classes like `NetworkDevice` and `Connection`. The `NetworkDevice` class is highlighted, showing its `increase_risk()` method.
- DEFENDER (03):** Contains the `Defender` class, which is currently empty.

The `NetworkDevice` class shows the following methods:

```

· increase_risk()
· inspect()
· block()
· unblock()
· isolate()
· restore()
· secure_connection()
· overload()
· isolate()
· block()
· unblock()
· lock_files()
· restore()
· intercept()
· secure()
· block()
· unblock()

```

The `CredentialAttack` class shows the following methods:

```

· perform_attack()
· execute()
· target.increase_risk(1)
· if self.tracer is not None:
    self.tracer.method_completed(
        self.name,
        "CredentialAttack",
        "execute",
        target.device_id
    )

```

The `NetworkDevice` class shows the following methods:

```

· self.tracer.state_changed(
    self._device_id,
    "NetworkDevice",
    "increase_risk",
    "risk_level",
    previous_value,
    self._risk_level
)
· self.tracer.method_completed(
    self._device_id,
    "NetworkDevice",
    "increase_risk"
)

```

The `Defender` class shows the following methods:

```

· inspect()
· block()
· unblock()
· isolate()
· restore()
· secure_connection()

```

The interface also includes a 'SYSTEM READY' status at the bottom right.

6.4 Test Case 4: DoS Attack

Test Input	Expected Result	Findings
Click <code>execute()</code> under <code>DosAttack</code> .	Simulation should start running. <code>DEV001</code> risk should increase and traffic should change from <code>NORMAL</code> to <code>HIGH</code> .	- The risk level increased correctly. <code>traffic_level</code> changed from <code>NORMAL</code> → <code>HIGH</code>. - The interaction path and affected object were highlighted. - The relevant Python source lines and Terminal events were shown.

Test Evidence

The screenshot displays a simulation interface for a cybersecurity scenario. At the top, there's a 'LOOP' section with playback controls (Previous, Next, Play, Pause, Replay, Reset) and a 'SCENARIO Cybersecurity' label. Below this is a 'TERMINAL' window showing the following events:

```

EVENT 8 / 8
MODE: Manual / Guided
OOF CONCEPTS: Encapsulation - Object State
06 STATE DEV001 - traffic_level
   NORMAL -> HIGH
07 DONE NetworkDevice.overload()
   DEV001
08 DONE DoSAttack.execute()
   DoS Attack -> DEV001
  
```

The main area shows a class hierarchy and state diagram. The 'ATTACKER' column (01) includes classes like `Attacker`, `CredentialAttack`, `MitMAttack`, `DoSAttack`, `DDoSAttack`, `MalwareAttack`, and `RansomwareAttack`. The 'VICTIM / DEVICE' column (02) includes `NetworkDevice` and `Connection`. The 'DEFENDER' column (03) includes `Defender`. The `DoSAttack` class is highlighted, showing its `execute()` method. The `NetworkDevice` class is also highlighted, showing its `increase_risk()` and `overload()` methods. The `Defender` class is also highlighted, showing its `inspect()`, `block()`, `unblock()`, `isolate()`, `restore()`, and `secure_connection()` methods. The state diagram shows the flow of the attack from the attacker to the victim device.

6.5 Test Case 5: DDoS Inheritance (DDoS)

Test Input	Expected Result	Findings
Click <code>execute()</code> under <code>DDoSAttack</code> .	Simulation should start running. <code>DDoSAttack.execute()</code> should call <code>DoSAttack.execute()</code> using <code>super()</code> before continuing its own behaviour. <code>DEV001</code> risk should increase and traffic should change from <code>NORMAL</code> to <code>CRITICAL</code> .	- The parent method call was shown visually. <code>super()</code> execution was highlighted. - The same <code>DDoSAttack</code> object remained active. - The interaction path and affected object were highlighted. - The relevant Python source lines and Terminal events were shown. - The Terminal explained the inheritance relationship.

Test Evidence

The screenshot displays a cyber security simulation interface with the following components:

- Terminal Window:** Shows the execution of `DDoSAttack.execute()` and the resulting state changes for `DEV001`, including `traffic_level` and `risk` updates.
- Class Hierarchy:** A tree view showing the relationship between `Attacker`, `NetworkDevice`, and `Defender` classes, with `DDoSAttack` inheriting from `DoSAttack`.
- Object List:** A list of active objects, including `Attacker`, `NetworkDevice`, and `Defender`, with their respective methods and attributes.

6.6 Test Case 6: Ransomware Inheritance and File Locking

Test Input	Expected Result	Findings
Click <code>execute()</code> under <code>RansomwareAttack</code> .	Simulation should start running. <code>MalwareAttack.execute()</code> should run first, followed by ransomware-specific behaviour. <code>DEV001</code> risk should increase and <code>files_locked</code> changed from <code>False</code> to <code>True</code> .	- The parent <code>MalwareAttack</code> method executed through <code>super()</code>. - The risk level increased. <code>files_locked</code> changed from <code>False</code> to <code>True</code>. - The interaction path and affected object were highlighted. - The relevant Python source lines and Terminal events were shown. - The Terminal explained the inheritance relationship.

Test Evidence

The screenshot displays the LOOP Cybersecurity simulation interface. At the top, there's a 'PLAYBACK' section with buttons for 'Previous', 'Next', 'Play', 'Pause', 'Replay', and 'Reset'. The main area is divided into three columns: 'ATTACKER', 'VICTIM / DEVICE', and 'DEFENDER'.

ATTACKER (01): Shows a list of attack classes. The `RansomwareAttack` class is selected, and its `execute()` method is being executed. The code snippet shows the method calling `super().execute(target)`, `target.increase_risk()`, and `target.lock_files()`.

VICTIM / DEVICE (02): Shows the state of the `NetworkDevice` class. The `lock_files()` method is being executed, which calls `self.tracer.state_changed()` to update the `files_locked` attribute from `False` to `True`. The `increase_risk()` method is also shown, which calls `self.tracer.method_completed()` to update the `risk` attribute.

DEFENDER (03): Shows the state of the `Defender` class. The `inspect()` method is being executed, which calls `self.tracer.method_completed()` to update the `files_locked` attribute from `False` to `True`.

The bottom of the interface shows a 'SYSTEM READY' status.

6.7 Test Case 7: Defender Restore

Test Input	Expected Result	Findings
Execute an attack first, then click <code>restore()</code> under Defender.	The affected device should return to its normal state.	- The Defender targeted the correct device. - The object indicator changed to the defense highlight. - Risk, traffic, isolation, and file-locking states were restored where applicable. - The relevant Python source lines were shown. - The Terminal showed the restore execution and resulting state changes.

Test Evidence

The screenshot displays the Cyberspace simulation interface during the 'Defender Restore' test case. The interface is divided into several sections:

- Terminal (Top):** Shows the execution of the `restore()` method on the `Defender` object. The output indicates that the `Defender` object is now `DEFENDER` and the `restore()` method is being executed.
- Attacker Panel (Left):** Lists various attack classes and their methods. The `CredentialAttack` class is highlighted, showing its `execute()` method. Other classes include `MitMAttack`, `DoSAttack`, `DDoSAttack`, `MalwareAttack`, and `RansomwareAttack`.
- Victim / Device Panel (Middle):** Shows the state of the `NetworkDevice` object. The `restore()` method is highlighted, showing its implementation. The `restore()` method calls `self.__tracer.state_changed()` with parameters for `self.__device_id`, `self.__device_id`, `NetworkDevice`, `restore`, `files_locked`, `previous_files_locked`, and `self.__files_locked`.
- Defender Panel (Right):** Shows the state of the `Defender` object. The `restore()` method is highlighted, showing its implementation. The `restore()` method calls `self.__tracer.state_changed()` with parameters for `self.__device_id`, `self.__device_id`, `NetworkDevice`, `restore`, `files_locked`, `previous_files_locked`, and `self.__files_locked`.

6.8 Test Case 8: Reset Simulation

Test Input	Expected Result	Findings
Click Reset after performing actions.	The backend and interface should return to their initial state.	• Device and connection states were reset. • Terminal history was cleared. • Active source and object highlights were removed. • Playback returned to 0/0.

Test Evidence

The screenshot displays a cybersecurity simulation interface in a 'LOOP' state. At the top, there are playback controls: 'PLAYBACK', 'Previous', 'Next', 'Play', 'Pause', 'Replay', and 'Reset'. The 'Reset' button is highlighted, and the progress is shown as '0 / 0'. The scenario is labeled 'Cybersecurity'.

The main interface is divided into three columns:

- ATTACKER (01):** Contains a list of attack classes and their methods. The classes are: `Attacker` (object: Attacker 1), `CredentialAttack` (object: Credential Attack), `MitMAttack` (object: MITM Attack), `DoSAttack` (object: DoS Attack), `DDoSAttack` (object: DDoS Attack), `MalwareAttack` (object: Malware Attack), and `RansomwareAttack` (object: Ransomware Attack). Each class has an `execute()` method.
- VICTIM / DEVICE (02):** Contains a list of device classes and their methods. The classes are: `NetworkDevice` (object: DEV001, DEV002) and `Connection` (object: CON001). The methods for `NetworkDevice` are: `increase_risk()`, `overload()`, `isolate()`, `block()`, `unblock()`, `lock_files()`, and `restore()`. The methods for `Connection` are: `intercept()`, `secure()`, `block()`, and `unblock()`.
- DEFENDER (03):** Contains a list of defender classes and their methods. The class is: `Defender` (object: Defender 1). The methods are: `inspect()`, `block()`, `unblock()`, `isolate()`, `restore()`, and `secure_connection()`.

The terminal at the top shows the text: 'Waiting for execution...' and 'Select a method to begin. The execution history will appear here.'

The bottom right corner of the interface shows 'SYSTEM READY'.

7. Challenges

Challenge	How it was handled
Frontend animations were initially separate from the real Python simulation.	FastAPI endpoints and ExecutionTracer events were used so the interface is now driven by actual Python execution.
Keeping playback controls synchronized with execution events was difficult.	Playback state was centralized so Previous, Next, Play, Pause, and Replay all use the same event sequence.
Previous playback did not restore source-code scrolling correctly.	Playback reconstruction was updated to remember and scroll to the last executed source line.
Reset stopped working correctly during playback development.	Reset was connected back to the real /api/reset endpoint and the playback state is cleared before rebuilding the interface.
The original JavaScript and CSS files became too large to maintain easily.	They were split into smaller files based on responsibility, such as playback, circuits, source display, panels, and effects.
It was difficult to visually distinguish a class from an actual object instance.	Object indicators such as DEV001, DEV002, and CON001 were added directly to the class cards.
Highlighting the whole device panel made it unclear which object was affected.	Attack and defense effects were moved to the individual object-instance chips.

8. Improvements

- Add more simulation scenarios besides cybersecurity, such as banking, smart homes, or e-commerce.
- Add a scenario selector so users can switch between different OOP examples.
- Improve support for smaller screens and mobile devices.
- Add more guided lessons focused on specific OOP concepts.
- Add clearer visual explanations for composition and polymorphism.
- Allow users to create or configure their own objects and scenarios.
- Improve accessibility, including keyboard navigation and reduced-motion options.